



Universidad del Valle de Guatemala
Facultad de Ingeniería
Departamento de Ciencias de la Computación
CC3067 — Redes

Proyecto 1: Uso de un protocolo existente (MCP)

Informe de avance — Entrega parcial

Nicolás Concuá
Carné: 23197

19 de agosto de 2026

8. Especificación de los servidores MCP

El proyecto consiste en un chatbot de consola que actúa como "anfitrión" (host) del protocolo Model Context Protocol (MCP). El anfitrión coordina varios clientes MCP que se comunican con servidores MCP locales. Una restricción central del proyecto es que el protocolo se implementa de forma manual sobre "JSON-RPC 2.0", sin utilizar ningún SDK de MCP (FastMCP, SDK oficial, etc.). Este informe documenta la especificación de los servidores locales integrados a la fecha de la entrega parcial.

8.1 Transporte y métodos del protocolo (endpoints)

Todos los servidores locales se comunican mediante el transporte "stdio": el anfitrión lanza cada servidor como un proceso hijo e intercambia mensajes JSON-RPC 2.0 (una línea de texto JSON por mensaje, delimitada por salto de línea) a través de stdin/stdout. Los métodos (endpoints) del protocolo implementados son:

- **initialize**: negociación inicial: versión del protocolo, capacidades e información del servidor.
- **notifications/initialized**: notificación del cliente indicando que está listo (sin respuesta).
- **tools/list**: el servidor devuelve la lista de herramientas disponibles y el esquema de sus parámetros.
- **tools/call**: el cliente invoca una herramienta con sus argumentos; el servidor la ejecuta y responde.
- **ping**: verificación de disponibilidad del servidor.

Cada petición lleva un identificador (id) que permite correlacionar la respuesta con su solicitud.

8.2 Servidor desarrollado: sugarmill (ingenio azucarero)

Es el servidor MCP desarrollado por el estudiante. Modela dos decisiones reales de un ingenio azucarero durante la zafra: primero, qué lotes de caña cortar según su "madurez" y la capacidad de molienda; y segundo, cuánto pagar al cañicultor según la "calidad" de la caña entregada. Está implementado a mano con la clase MCPServer del proyecto.

- **Ejecución**: `python -m src.servers.sugarmill`
- **Transporte**: JSON-RPC 2.0 sobre stdio.
- **Datos**: simulados en `data.json`; las muestras de laboratorio se mantienen en memoria durante la sesión.

Herramientas y parámetros:

Herramienta	Parámetros	Descripción
list_fields	status?	Lista los lotes de caña (variedad, área, pol, brix, estado).
field_maturity	field_id, weeks?	Proyecta el pol (sacarosa) de un lote en las próximas semanas.
recommend_harvest_plan	weekly_capacity_tons	Recomienda qué lotes cortar para maximizar azúcar recuperable.

estimate_sucrose_loss	field_id, hours_since_cut	Estima la sacarosa perdida tras el corte o quema.
cane_quality	pol, brix	Calcula pureza y azúcar recuperable (KATC).
register_lab_sample	field_id, pol, brix	Registra una muestra de laboratorio.
list_lab_samples	(ninguno)	Lista las muestras registradas en la sesión.
compute_payment	tons, price_per_ton, pol, brix	Calcula el pago al productor según tonelaje y calidad.
zafra_report	(ninguno)	Resume KPIs de la zafra (TCH, TAH, azúcar recuperable).

Fórmulas del dominio:

Concepto	Fórmula
Pureza (%)	$\text{pol} / \text{brix} * 100$
Azúcar recuperable KATC (kg/ton)	$\text{pol} * 10 * 0.88$ (eficiencia de recobrado 0.88)
Factor de calidad	$\text{KATC} / 112$ (112 kg/ton de referencia)
Pago al productor	$\text{tons} * \text{price_per_ton} * \text{factor de calidad}$
Proyección de madurez	subida lineal a peak_pol, luego -0.15 pol por semana
Deterioro tras el corte	$\text{pol} * (1 - 0.002 * \text{horas})$
TCH / TAH	ton de caña por hectárea / ton de azúcar por hectárea

Nota: son modelos simplificados pero plausibles a nivel de industria, adecuados para una simulación del curso.

8.3 Servidores oficiales integrados (locales)

Además del servidor propio, el anfitrión integra dos servidores MCP oficiales que se ejecutan localmente. Son procesos externos que nuestro código nunca importa: únicamente se comunican con ellos mediante el cliente JSON-RPC escrito a mano.

- **Filesystem:** se ejecuta con npx y el paquete server-filesystem. Expone 14 herramientas (read_file, write_file, list_directory, etc.), restringido al directorio de trabajo.
- **Git:** se ejecuta con python y el módulo mcp_server_git. Expone 12 herramientas (git_status, git_add, git_commit, git_log, etc.). Requiere fijar la dependencia "mcp<2".

10. Conclusiones y comentarios

En esta primera entrega se implementó un chatbot de consola que funciona como anfitrión MCP y cumple las funcionalidades 1 a 5 del proyecto: conexión con un LLM vía API, mantenimiento del contexto de la sesión, registro (log) de todas las interacciones con los servidores, uso de los servidores oficiales Filesystem y Git, y un servidor MCP propio para un caso de industria (ingenio azucarero). El protocolo MCP se implementó de forma manual sobre JSON-RPC 2.0, tanto del lado del cliente como del servidor propio.

Dificultades y cómo se resolvieron

- El servidor Git oficial finalizaba al iniciar si se le indicaba un repositorio inexistente; se resolvió no pasando la ruta por defecto y dejando que el anfitrión inicialice el repositorio del directorio de trabajo.
- La dependencia "mcp" se instalaba en su versión 2.0, incompatible con el servidor Git (que usa la API 1.x); se fijó la versión "mcp<2".
- Los esquemas de las herramientas incluían claves de JSON Schema que el LLM rechazaba; se agregó un saneamiento de esquemas antes de enviarlos.
- El LLM devolvía errores temporales (HTTP 503) por saturación; se hizo la sesión resiliente para que no se caiga y permita reintentar sin perder el contexto.
- El modelo configurado inicialmente ya no estaba disponible para cuentas nuevas; se cambió a un alias de modelo vigente.

Lecciones aprendidas

- Separar la capa del LLM de la capa del protocolo permitió cambiar de proveedor (de Anthropic a Gemini) modificando un solo archivo, sin tocar el cliente ni los servidores MCP.
- Implementar JSON-RPC 2.0 a mano (framing, handshake, correlación id-respuesta y manejo de notificaciones) aclaró cómo funciona MCP por debajo de los SDKs.
- Los servidores MCP son intercambiables: el mismo cliente genérico habla con servidores oficiales y con el servidor propio sin cambios.

Comentario final

El proyecto muestra de forma tangible cómo un LLM, por sí solo limitado a su conocimiento, se convierte en un agente capaz de ejecutar acciones reales (crear archivos, versionar en Git y consultar la operación de un ingenio) a través de un protocolo estándar. La base construida en esta entrega deja lista la migración del servidor propio a un despliegue remoto y su posterior análisis con Wireshark para la segunda entrega.